

Big Data Analytics

Big Data Analytics — Study Notes

Table of Contents

1. [Foundations of Big Data](#)
 2. [Big Data Architecture](#)
 3. [Distributed Computing Concepts](#)
 4. [Hadoop Ecosystem](#)
 5. [MapReduce](#)
 6. [Big Data Storage Technologies](#)
 7. [Processing Frameworks](#)
 8. [Data Ingestion Tools — Kafka & Sqoop](#)
 9. [Query & Analytics Tools — Hive & Pig](#)
 10. [Data Lakes & Data Warehouses](#)
 11. [Big Data Visualization](#)
 12. [Machine Learning for Big Data](#)
-

1. Foundations of Big Data

1.1 What is Big Data?

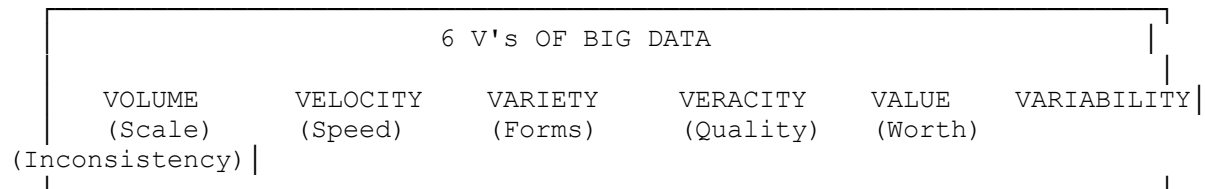
Big Data refers to extremely large and complex datasets that **cannot be stored, processed, or analysed using traditional database tools** like MySQL or Excel within a reasonable time.

Analogy: A traditional database is like a single filing cabinet — great for a small office. Big Data is like managing the records of an entire country. You need a completely different system — distributed storage rooms, many clerks working in parallel, and smart lookup mechanisms.

The goal of Big Data Analytics is to examine these massive datasets to **uncover hidden patterns, correlations, trends, and insights** that help organisations make better decisions.

1.2 The 6 V's of Big Data

The 6 V's describe the defining characteristics of Big Data:



V	Definition	Example
Volume	Sheer amount of data generated	Facebook generates ~500 TB/day
Velocity	Speed at which data is generated and must be processed	Stock prices update every millisecond
Variety	Different formats — structured, semi-structured, unstructured	SQL tables + JSON logs + social media images
Veracity	Trustworthiness and quality of data	Twitter data has bots, typos, missing values
Value	Business worth extracted from data	Walmart found Pop-Tarts sell before hurricanes
Variability	Data meaning changes over time or context	The word "bank" means finance or river shore

1.3 Types of Big Data

Structured Data

- Highly organised, fits neatly into rows and columns.
- Stored in relational databases (MySQL, Oracle).
- Example: Customer table with `id`, `name`, `email`, `age`.

Semi-Structured Data

- Has some structure but not rigid schema. Uses tags or markers.
- Example: JSON, XML, CSV, Log files.

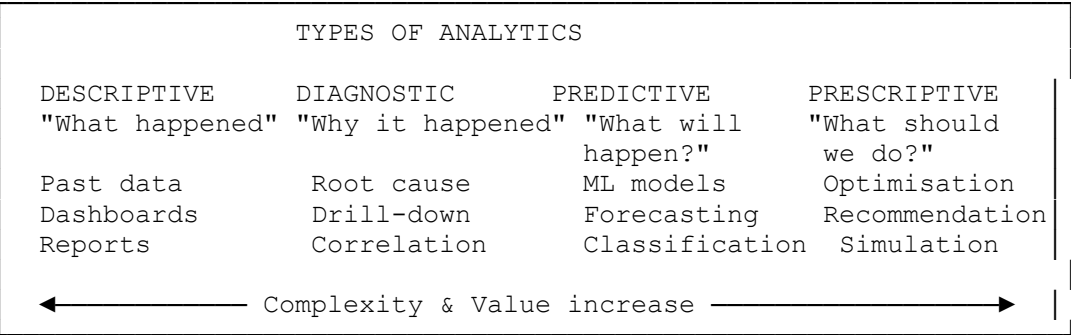
```
{
  "user_id": 101,
  "event": "purchase",
  "items": ["laptop", "mouse"],
  "timestamp": "2024-01-15T14:30:00"
}
```

Unstructured Data

- No predefined format. Hardest to process and analyse.

- Makes up ~80% of enterprise data.
- Example: Videos, images, audio, emails, PDFs, social media posts.

1.4 Types of Big Data Analytics



- Descriptive:** Summarises historical data. *"Sales were \$5M last quarter."*
- Diagnostic:** Explains why something happened. *"Sales dropped because a competitor launched a discount."*
- Predictive:** Forecasts future outcomes using statistical models. *"Sales will grow 12% next quarter."*
- Prescriptive:** Recommends actions to achieve a desired outcome. *"Offer a 10% discount to at-risk customers."*

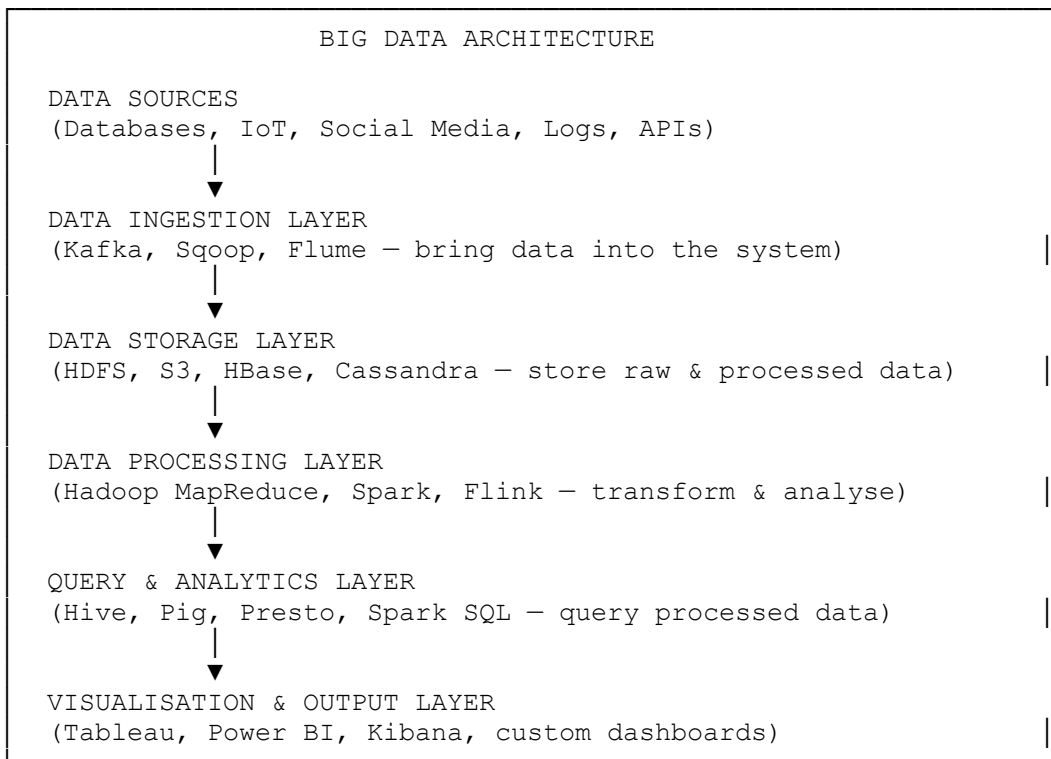
1.5 Traditional Data vs Big Data

Aspect	Traditional Data	Big Data
Volume	Gigabytes	Terabytes to Petabytes
Data type	Structured only	Structured + Semi + Unstructured
Storage	RDBMS (single server)	Distributed systems (HDFS, S3)
Processing	SQL queries on one machine	Parallel processing (Hadoop, Spark)
Schema	Schema defined before storing (schema-on-write)	Schema applied at query time (schema-on-read)
Scaling	Vertical (bigger machine)	Horizontal (more machines)
Cost	Expensive enterprise hardware	Commodity hardware (cheap)
Real-time	Limited	Native (Kafka, Flink, Spark Streaming)

Analogy: Traditional data processing is like cooking a meal in your kitchen — works fine for a family dinner. Big Data is like catering for a stadium — you need a commercial kitchen, many chefs, parallel cooking stations, and a logistics system.

2. Big Data Architecture

A Big Data system has several layers, each responsible for a specific function:



2.1 Batch Processing vs Stream Processing

These are the two fundamental modes of processing data in a Big Data system.

BATCH PROCESSING:

Data accumulates over time → processed all at once at scheduled intervals

[Data] → [Accumulate] → [Process entire batch] → [Output]

Monday's orders → collected all day → processed at midnight → report next morning

STREAM PROCESSING:

Data processed immediately as it arrives – event by event or in tiny windows

[Data] → [Process instantly] → [Output in real time]
Credit card swipe → fraud check within 200ms → approve or decline

Aspect	Batch Processing	Stream Processing
Latency	Minutes to hours	Milliseconds to seconds
Data size	Processes large historical datasets	Processes live, recent data
Complexity	Simpler to build and debug	More complex (handles out-of-order data)
Use cases	Payroll, month-end reports, ETL	Fraud detection, live dashboards, alerts
Tools	Hadoop MapReduce, Spark Batch	Kafka, Flink, Spark Streaming

Analogy: Batch is like washing all your clothes on Sunday — efficient, but you can't wear fresh clothes on Tuesday. Streaming is like a dry cleaner who cleans each garment the moment you drop it off.

2.2 Data Ingestion

Data ingestion is the process of **collecting and importing raw data from various sources into a storage or processing system.**

Two types:

- **Batch ingestion:** Data collected at scheduled intervals (hourly, daily). Example: Sqoop imports a MySQL table to HDFS every night at 2 AM.
- **Real-time ingestion:** Data flows continuously as events occur. Example: Kafka captures every click on a website in real time.

Common ingestion tools:

- **Apache Kafka:** For high-throughput real-time event streaming.
 - **Apache Sqoop:** For bulk import/export between RDBMS and HDFS.
 - **Apache Flume:** For collecting log data from servers into HDFS.
-

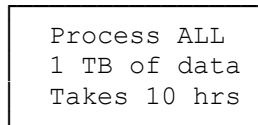
3. Distributed Computing Concepts

3.1 What is Distributed Computing?

Distributed computing is a model where **multiple computers work together as a single system** to solve a problem. Each machine handles a portion of the work in parallel.

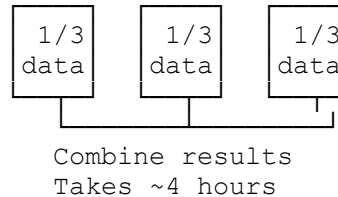
WITHOUT DISTRIBUTED COMPUTING:

One big machine



WITH DISTRIBUTED COMPUTING:

Many small machines



Key benefit: Adding more machines (horizontal scaling) increases capacity linearly and cost-effectively, unlike buying one very expensive server.

3.2 Parallel Data Processing

Parallel processing means **splitting a task into sub-tasks and executing them simultaneously** across multiple processors or nodes.

Types:

- **Data Parallelism:** Same operation applied to different chunks of data simultaneously. MapReduce and Spark use this model.
- **Task Parallelism:** Different operations run concurrently. Example: ingestion and transformation happening at the same time in a pipeline.

DATA PARALLELISM EXAMPLE — Computing sum of 1 million numbers:

Without: [1, 2, 3, ..., 1,000,000] → Node 1 adds all → result
Time: T

With parallelism:

```
Node 1: sum([1...250,000]) = 31,250,125,000
Node 2: sum([250,001...500,000]) = 93,750,375,000
Node 3: sum([500,001...750,000]) = 156,250,375,000
Node 4: sum([750,001...1,000,000]) = 218,750,375,500
Final: 31,250,125,000 + 93,750,375,000 + ... = 500,000,500,000
Time: T/4
```

3.3 Fault Tolerance

In a distributed system with hundreds of nodes, hardware failures are **normal and expected** — a node crashing is not an exception, it is routine.

Fault tolerance is the system's ability to **continue operating correctly even when some components fail**.

How Big Data systems achieve fault tolerance:

1. Data Replication

Each piece of data is copied to multiple nodes. If one node fails, another holds the same data.

```
Block B1 (128MB of data):  
Copy 1 → Node 5  
Copy 2 → Node 12    ← if Node 5 fails, copies 2 & 3 still exist  
Copy 3 → Node 3
```

2. Task Re-execution

If a processing task fails (node crash mid-job), the framework automatically re-runs it on another healthy node. The input data is still available on HDFS.

3. Heartbeat Monitoring

Each node regularly sends a "heartbeat" signal to the master. If the master stops receiving heartbeats, it marks that node as dead and reassigns its work.

3.4 Replication in Distributed Systems

Replication means **storing the same data on multiple nodes** to ensure availability and durability.

```
HDFS Default Replication Factor = 3:  
  
File: sales_data.csv (split into 3 blocks B1, B2, B3)  
  
Node 1:  [B1] [B3]  
Node 2:  [B1] [B2]      Each block stored on 3 nodes.  
Node 3:  [B2] [B3]      System survives 2 node failures per block.  
Node 4:  [B1] [B2] [B3]
```

Rack-aware replication: HDFS places replicas on different server racks so that even if an entire rack loses power, data survives on another rack.

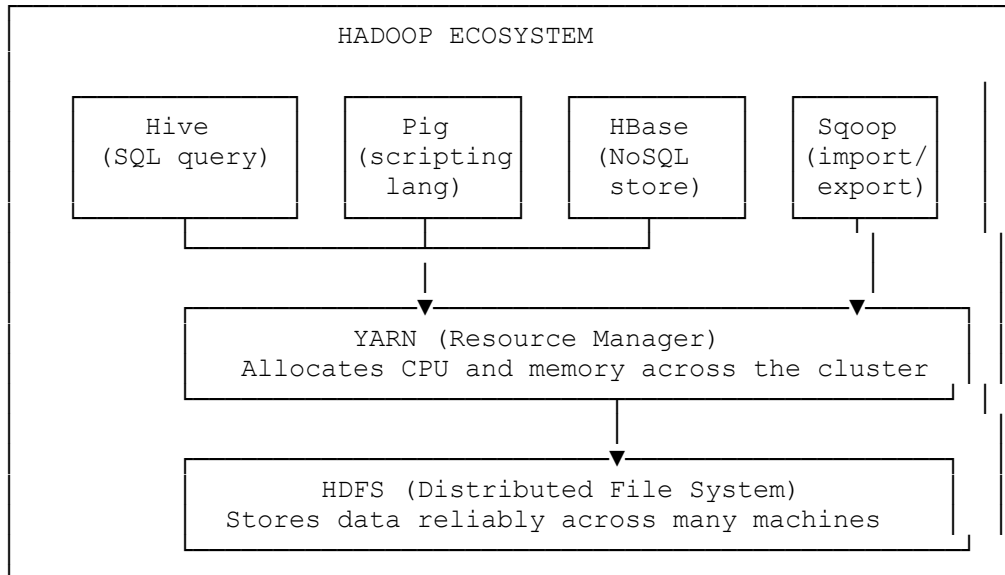
Tradeoff: Higher replication factor = more storage used but better fault tolerance and read performance (more copies = more nodes you can read from).

4. Hadoop Ecosystem

4.1 What is Hadoop?

Apache Hadoop is an **open-source framework for storing and processing very large datasets across clusters of commodity computers** using a simple programming model.

Core components:



4.2 HDFS — Hadoop Distributed File System

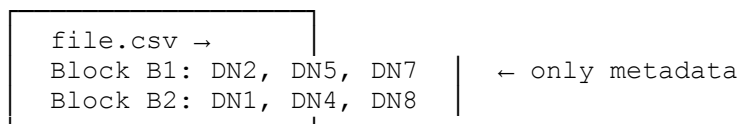
HDFS is the **storage backbone of Hadoop**. It stores large files by splitting them into blocks and distributing them across many DataNodes.

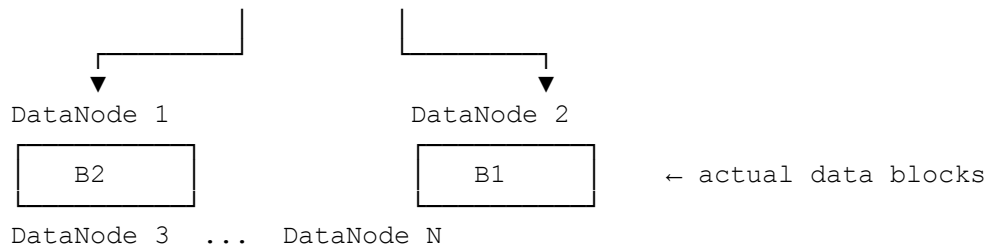
Key Concepts:

Concept	Description
Block	Default size 128 MB. Large files split into multiple blocks.
NameNode	The master — stores metadata (file names, which blocks are where). Does NOT store actual data.
DataNode	The workers — store the actual data blocks. Many DataNodes per cluster.
Replication	Each block stored on 3 nodes by default for fault tolerance.

HDFS Architecture:

NameNode (Master)





How a file write works:

1. Client tells NameNode it wants to write a file.
2. NameNode assigns DataNodes for each block (e.g., 3 nodes per block).
3. Client writes Block 1 to DataNode A → DataNode A copies to DataNode B → B copies to C (pipeline).
4. NameNode updates its metadata once all copies are confirmed.

How a file read works:

1. Client asks NameNode where blocks of the file are stored.
2. NameNode returns the list of DataNodes for each block.
3. Client reads directly from the nearest DataNode — NameNode not involved in data transfer.

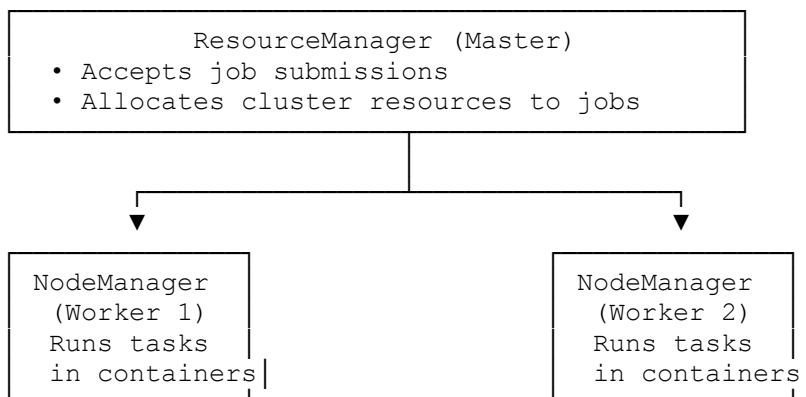
Why large block size (128 MB)?

Larger blocks mean fewer blocks to track in NameNode metadata, and they enable high-throughput sequential reads — ideal for MapReduce jobs that read the entire file anyway.

4.3 YARN — Yet Another Resource Negotiator

YARN is the **resource management layer** of Hadoop. It decides how CPU and memory are distributed across jobs running on the cluster.

YARN Components:



Application
Master

← One per job (e.g., MapReduce AM, Spark AM)
Negotiates resources, tracks task progress

YARN separates:

- **Resource management** (ResourceManager) from
- **Job management** (ApplicationMaster, one per job)

This allows Hadoop to run **multiple frameworks** (MapReduce, Spark, Flink) on the same cluster simultaneously — each gets its own ApplicationMaster.

4.4 Hadoop 2.x vs Hadoop 3.x

Feature	Hadoop 2.x	Hadoop 3.x
Default block size	128 MB	256 MB
Replication	3x (default)	Erasure Coding (more space-efficient)
NameNode HA	Active/Standby (2 NN)	Multiple Standby NNs
YARN timeline	v1	v2 (scalable, more metrics)
Storage overhead	200% extra for 3x replication	~50% with erasure coding
Min Java	Java 7	Java 8

Erasure Coding (Hadoop 3.x key feature):

Instead of storing 3 full copies of data, erasure coding stores the original data + mathematical "parity" blocks. You can reconstruct lost data from the parity without having full copies.

3x Replication: Store 1 TB → uses 3 TB of disk
Erasure Coding (RS-6-3): Store 1 TB → uses only ~1.5 TB
Saves ~50% storage at the cost of more CPU for reconstruction.
Used for cold/archival data where reads are infrequent.

5. MapReduce

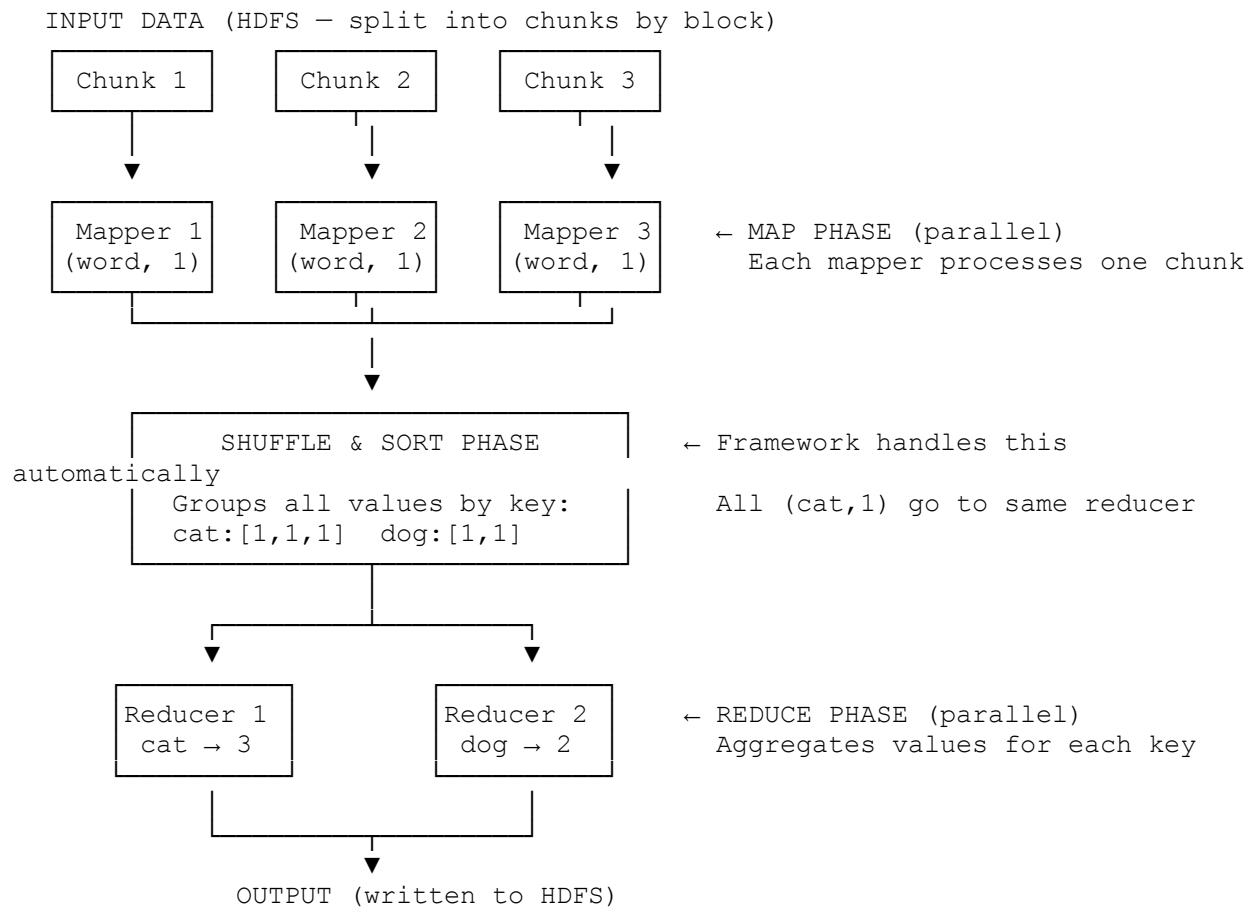
5.1 Core Concept

MapReduce is a **programming model for processing large datasets in parallel** across a cluster. It breaks any big computation into two simple steps: **Map** (transform) and **Reduce** (aggregate).

Analogy — Counting words in 1000 books:

You hire 100 students. Each student gets 10 books and writes down every word they see with a tally mark (Map). Then you group all tally marks for the same word together and one person per word adds them up (Reduce). You'd finish in hours instead of years.

5.2 MapReduce Phases



5.3 Mapper

The **Mapper** takes the raw input and produces intermediate key-value pairs.

- Input: a chunk of the data (e.g., one line of a text file).
- Output: zero or more (key, value) pairs.

Word Count Mapper:

Input line: "the cat sat on the mat"

```
Output:
(the, 1)
(cat, 1)
(sat, 1)
(on, 1)
(the, 1)    ← same word emitted again
(mat, 1)
```

The mapper **does not aggregate** — it just transforms. Each word gets a count of 1 regardless of how many times it appears in that line. The framework handles the grouping.

5.4 Reducer

The **Reducer** receives all values for a given key and produces the final aggregated output.

- Input: one key and a list of all values for that key from all mappers.
- Output: final (key, result) pairs.

Word Count Reducer:

```
Input:  (cat, [1, 1, 1])
Output: (cat, 3)

Input:  (dog, [1, 1])
Output: (dog, 2)
```

What happens between Mapper and Reducer — the Shuffle:

The framework automatically:

1. **Partitions** mapper output — each reducer gets a specific subset of keys.
2. **Sorts** the intermediate data by key.
3. **Transfers** data over the network from mapper nodes to reducer nodes.
4. **Groups** all values for the same key together before calling reduce().

This shuffle phase is often the **performance bottleneck** because it involves network transfer.

5.5 Combiner (Mini-Reducer)

A Combiner is an **optional local aggregation** that runs on the mapper node *before* the shuffle, reducing the amount of data sent over the network.

```
Without Combiner:
Mapper 1 emits: (cat,1) (cat,1) (cat,1) (dog,1) (cat,1)
Network sends 5 records.
```

With Combiner:

Mapper 1 locally aggregates: (cat,4) (dog,1)

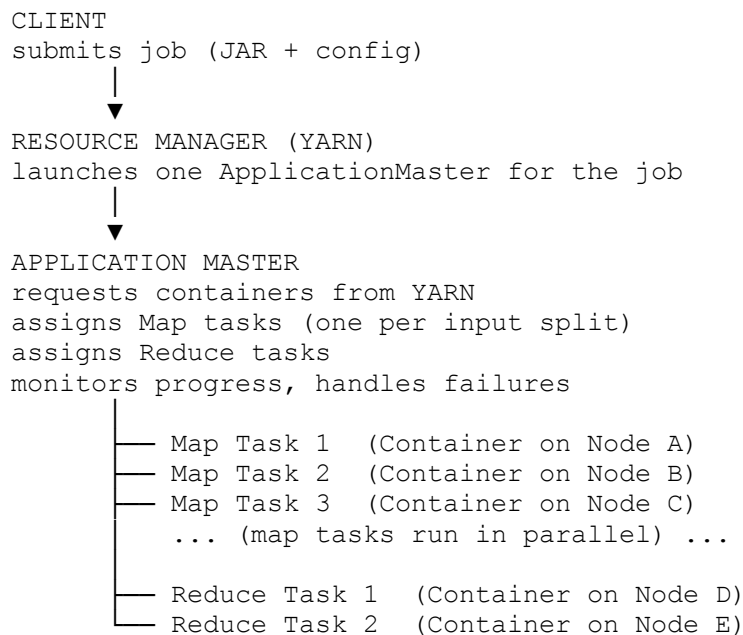
Network sends only 2 records. ← 60% less network traffic

Rule: Combiner is only valid for operations where partial aggregation gives the same final result:

✓ Sum, Count, Max, Min (associative & commutative)

X Average (average of averages ≠ overall average)

5.6 MapReduce Architecture



Data locality: YARN tries to schedule Map tasks on the **same node** that stores the input data block. This avoids sending data over the network — the computation goes to the data, not the other way around.

6. Big Data Storage Technologies

6.1 HDFS (Covered in Section 4.2)

Best for: storing large files for batch processing with Hadoop/Spark.

6.2 HBase

HBase is a **NoSQL, column-oriented database built on top of HDFS**. Unlike HDFS (which is a file system), HBase provides real-time random read/write access to individual rows.

HBase Data Model:

Row Key	Column Family: info	Column Family: scores
student:01	name: Alice, age: 21	math: 88, science: 92
student:02	name: Bob	math: 76
student:03	name: Carol, age: 20	(no scores stored yet)

Key concepts:

- **Row Key:** uniquely identifies a row. Rows are sorted by row key — design it well for your access pattern.
- **Column Family:** a group of related columns. Defined at table creation time.
- **Column Qualifier:** the actual column name within a family.
- **Versioning:** each cell stores multiple timestamped versions. You can retrieve the value at a specific time.

When to use HBase vs HDFS:

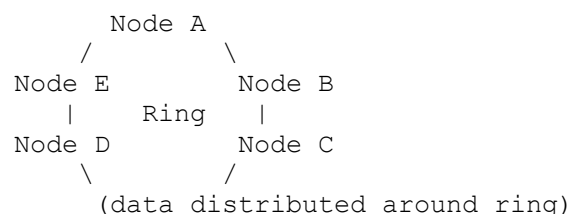
	HDFS	HBase
Access pattern	Sequential scan of entire file	Random read/write by row key
Latency	High (batch)	Low (milliseconds)
Use case	Log analysis, data lake	User profiles, real-time lookups

Analogy: HDFS is like a giant library where you read whole books. HBase is like a dictionary — you look up one word instantly.

6.3 Apache Cassandra

Cassandra is a **distributed NoSQL database** designed for massive write throughput and high availability with **no single point of failure**.

Cassandra Ring Architecture (no master node):



Any node can accept a read or write request.
No master — eliminates single point of failure.
Data automatically distributed via consistent hashing.

Key features:

- **Tunable consistency:** You choose how many nodes must confirm a write/read. More nodes = stronger consistency but slower.
- **Best for:** IoT sensor data, time-series, write-heavy workloads, applications needing 24/7 uptime.
- **CAP theorem position:** Cassandra favours **Availability + Partition Tolerance** — it will still accept writes even if some nodes are unreachable, syncing later.

6.4 MongoDB

MongoDB is a **document-oriented NoSQL database** that stores data as flexible JSON-like documents.

```
{
  "_id": "order_1001",
  "customer": "Alice",
  "items": [
    { "product": "Laptop", "qty": 1, "price": 999.99 },
    { "product": "Mouse", "qty": 2, "price": 29.99 }
  ],
  "total": 1059.97,
  "status": "shipped"
}
```

Why MongoDB for Big Data?

- Schema-flexible — different documents in the same collection can have different fields.
- Horizontal sharding — automatically distributes data across multiple servers.
- Rich query language — filter, aggregate, and transform JSON data.
- **Best for:** product catalogs, content management, user profiles, any data that is naturally document-shaped.

6.5 Comparing Storage Technologies

CHOOSE BASED ON ACCESS PATTERN:

Need to store raw files for batch analytics?	→ HDFS
Need real-time row lookup by key?	→ HBase
Need high-speed writes, always available?	→ Cassandra
Need flexible schemas, JSON documents?	→ MongoDB

Need SQL transactions on structured data?	→ RDBMS (MySQL, PostgreSQL)
Need analytics on structured warehouse data?	→ Hive / Redshift / BigQuery

7. Processing Frameworks

7.1 Apache Spark

Spark is a **fast, general-purpose distributed processing framework** that overcomes MapReduce's limitations by processing data **in memory** rather than writing intermediate results to disk after every step.

MapReduce (disk after each step):
Input → Map → [DISK] → Shuffle → [DISK] → Reduce → [DISK] → Output

Spark (in memory, disk only at end):
Input → Transform → Transform → Transform → Output
(all held in memory across steps – much faster)

Speed difference: Spark is typically **10x faster** for batch jobs and **100x faster** for iterative algorithms (like machine learning) compared to MapReduce.

Spark Core Concepts:

RDD (Resilient Distributed Dataset)

The fundamental data structure. An RDD is a fault-tolerant, immutable, distributed collection of objects spread across the cluster.

```
sales_rdd = sc.textFile("hdfs:///data/sales.csv")    # load from HDFS

filtered = sales_rdd.filter(lambda x: "Electronics" in x)    #
Transformation (lazy)
counted = filtered.count()                                   # Action
(triggers execution)
```

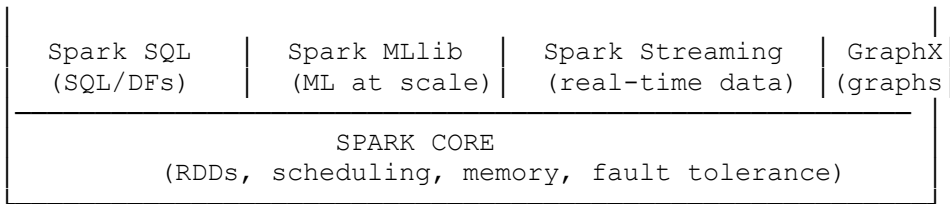
Two types of operations:

- **Transformations** (lazy — don't execute immediately): map, filter, groupBy, join
- **Actions** (trigger actual computation): count, collect, save, show

Lazy evaluation means Spark **builds a plan** of all transformations first, then optimises and executes them all at once when an action is called.

Spark Components:

SPARK STACK



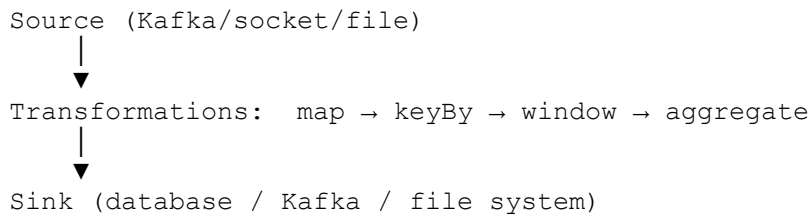
7.2 Apache Flink

Flink is a **true streaming processing framework** built for **low-latency, stateful processing of real-time data streams**.

Key difference from Spark Streaming:

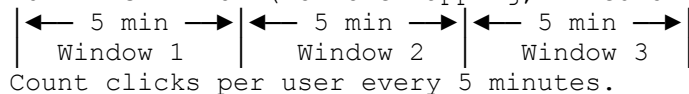
- Spark Streaming uses **micro-batching** — collects data for a few seconds, then processes as a small batch. There is always a small delay equal to the batch interval.
- Flink is **event-driven** — processes each event the instant it arrives, with millisecond latency.

FLINK JOB STRUCTURE:

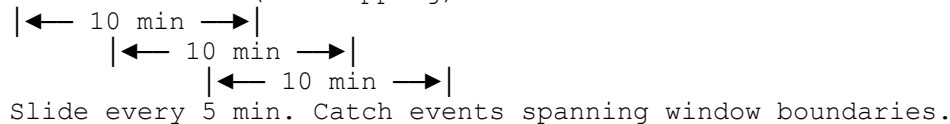


Flink's strength — Time Windows:

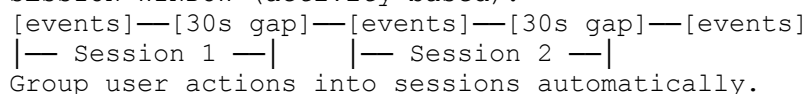
TUMBLING WINDOW (non-overlapping, fixed 5 min):



SLIDING WINDOW (overlapping):



SESSION WINDOW (activity-based):



Best for: real-time fraud detection, live dashboards, alerting systems, complex event processing.

7.3 Spark vs Flink vs MapReduce

PROCESSING FRAMEWORK COMPARISON			
Feature	MapReduce	Spark	Flink
Processing model	Batch only	Batch + quasi stream	True streaming + batch
Speed	Slow (disk)	Fast (memory)	Fastest
Latency	Minutes-hrs	Seconds	Milliseconds
Ease of use	Complex Java	Simple Python	Java/Scala/Py
ML support	Poor	MLlib (great)	FlinkML
Best for	Legacy batch	General BDA	Real-time jobs

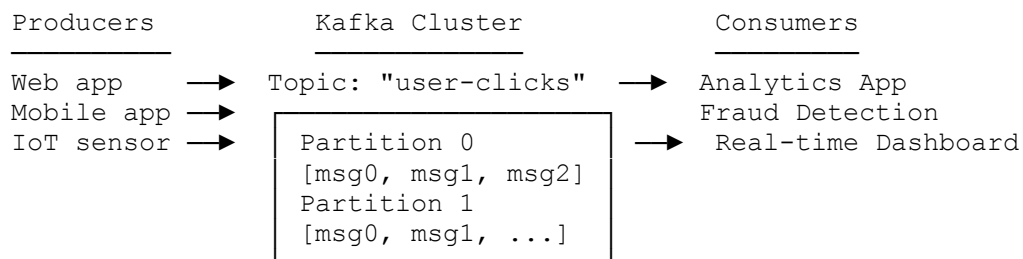
8. Data Ingestion Tools — Kafka & Sqoop

8.1 Apache Kafka

Kafka is a **distributed, high-throughput, fault-tolerant message streaming platform** used to build real-time data pipelines.

Analogy: Kafka is like a high-speed postal system for data. Senders (Producers) drop messages into post boxes (Topics). The postal service (Kafka) stores them reliably and delivers them to whoever subscribed (Consumers), at their own pace.

KAFKA ARCHITECTURE:



Key Concepts:

Term	Meaning
Producer	Application that sends/publishes messages to Kafka

Consumer	Application that reads/subscribes to messages from Kafka
Topic	A named category/feed to which messages are published
Partition	A topic is split into partitions for parallelism and scalability
Offset	Position of a message within a partition (like a line number)
Broker	A Kafka server. A cluster has multiple brokers for fault tolerance
Consumer Group	A group of consumers that share the work of reading a topic

Why partitions?

Topic "orders" with 4 partitions → 4 consumers can read in parallel
 More partitions = higher throughput

Consumer Group:

Partition 0 → Consumer A

Partition 1 → Consumer B

Partition 2 → Consumer C

Partition 3 → Consumer D

Each partition consumed by exactly one consumer in the group at a time.

Message retention: Kafka stores messages on disk for a configurable period (default 7 days). Unlike a traditional queue, messages are NOT deleted after consumption — multiple consumers can read the same message independently, at their own pace. You can even replay from the beginning.

8.2 Apache Sqoop

Sqoop is a tool for **bulk importing/exporting data between relational databases (RDBMS) and HDFS.**



How it works:

Sqoop divides the import query into n parallel SQL queries (one per mapper in a MapReduce job), each reading a different range of the table. All mappers run in parallel, writing their portion to HDFS.

```
sqoop import \  
  --connect jdbc:mysql://server/sales_db \  
  --table orders \  
  --target-dir /user/hadoop/orders \  
  --num-mappers 4          ← 4 parallel import tasks
```

Under the hood:

```
Mapper 1: SELECT * FROM orders WHERE id BETWEEN 1 AND 25000  
Mapper 2: SELECT * FROM orders WHERE id BETWEEN 25001 AND 50000  
Mapper 3: SELECT * FROM orders WHERE id BETWEEN 50001 AND 75000  
Mapper 4: SELECT * FROM orders WHERE id BETWEEN 75001 AND 100000
```

8.3 Apache ZooKeeper

ZooKeeper is a **centralised coordination service** for distributed systems. It stores small pieces of configuration and state information that all nodes in a cluster need to agree on.

What ZooKeeper provides:

- **Naming:** "Which node is the current Kafka leader for partition 3?"
- **Configuration:** "What is the current replication factor setting?"
- **Synchronisation:** "Only one node should act as HBase master at a time."
- **Leader Election:** When the current leader crashes, ZooKeeper helps elect a new one automatically.

Analogy: ZooKeeper is like the staff noticeboard in an office. Everyone checks it to find out who's in charge today, where the meeting room is, and what the current policies are. When the manager changes, the noticeboard is updated and everyone switches to the new manager.

Used by: Hadoop (NameNode HA), HBase (master election), Kafka (cluster metadata), Storm (topology coordination).

9. Query & Analytics Tools — Hive & Pig

9.1 Apache Hive

Hive is a **data warehouse tool built on top of Hadoop that lets you query large datasets using a SQL-like language called HiveQL (HQL).**

Analogy: MapReduce is like writing machine code — powerful but tedious. Hive is like writing Python — you express what you want at a high level, and the system figures out how to execute it in MapReduce/Tez/Spark underneath.

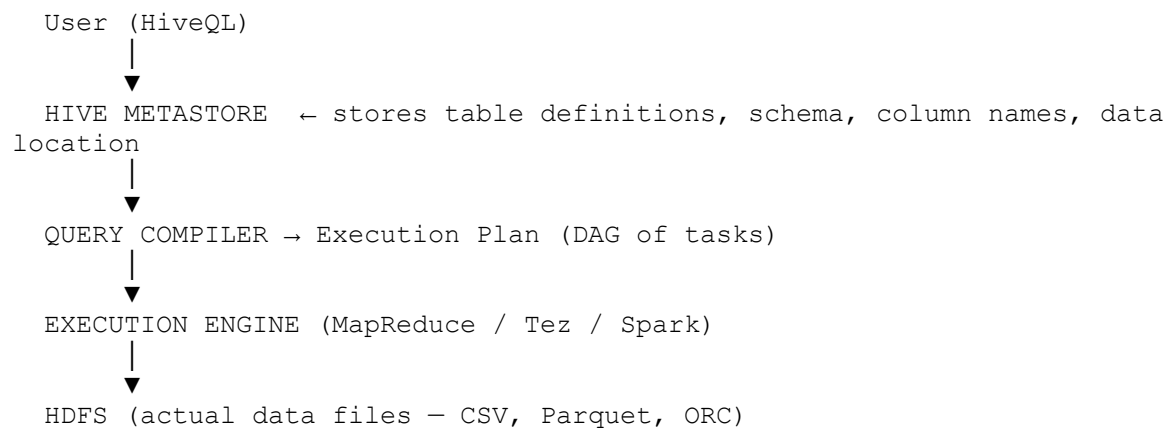
HiveQL Query:

```
SELECT department, AVG(salary) as avg_salary
FROM employees
WHERE year = 2024
GROUP BY department
ORDER BY avg_salary DESC;
```

What Hive does:

1. Parses and validates the HQL query
2. Converts it into a DAG of MapReduce/Tez jobs
3. Submits those jobs to the cluster
4. Returns results to the user — as if it were a database

Hive Architecture:



Hive Metastore is critical — it's a relational database (often MySQL) that stores:

- Table names and column definitions
- Where on HDFS the data files are located
- Partition information
- Statistics for query optimisation

Partitioning in Hive:

Without partitioning:

```
SELECT * FROM sales WHERE year=2024 AND month=3
```

→ Scans ALL data (every year and month) → slow

With partitioning:

```
CREATE TABLE sales (...) PARTITIONED BY (year INT, month INT);
```

→ Data stored as: /warehouse/sales/year=2024/month=3/data.parquet
 → Query only reads relevant partition folder → much faster

Bucketing:

Further divides partitions into a fixed number of files based on a hash of a column. Useful for efficient joins (two tables bucketed on the same join column → no full shuffle needed).

9.2 Hive Features & Limitations

Features:

- SQL-like syntax — easy for analysts who know SQL.
- Works on structured and semi-structured data.
- Supports partitioning, bucketing, indexing.
- Multiple storage formats: TextFile, SequenceFile, ORC, Parquet.
- User-Defined Functions (UDFs) for custom logic.

Limitations:

- **High latency:** Hive queries typically take minutes — not suitable for interactive/real-time queries.
 - **No row-level updates/deletes** (traditionally) — designed for append-only analytics, not OLTP.
 - **Not suitable for very low-latency queries** — use Presto/Impala for that.
-

9.3 Apache Pig

Pig is a **high-level data flow scripting platform** for processing large datasets on Hadoop. It uses a language called **Pig Latin**.

```
PIG LATIN EXAMPLE — Find top 5 pages by visit count:

raw_logs = LOAD 'hdfs:///logs/access.log' USING PigStorage(' ')
          AS (ip:chararray, date:chararray, page:chararray);

grouped = GROUP raw_logs BY page;

page_counts = FOREACH grouped GENERATE
               group AS page,
               COUNT(raw_logs) AS visits;

sorted = ORDER page_counts BY visits DESC;
top5 = LIMIT sorted 5;
STORE top5 INTO 'hdfs:///output/top_pages';
```

Pig compiles this script into a series of MapReduce jobs automatically.

Pig vs Hive:

Aspect	Hive	Pig
Language	HQL (SQL-like)	Pig Latin (data flow language)
Who uses it	SQL-savvy analysts	Developers, data engineers

Best for	Ad-hoc querying, reporting	ETL pipelines, data transformation
Structure required	Table with defined schema	Can work with raw, messy data
Paradigm	Declarative ("what you want")	Procedural ("how to do it step by step")

9.4 Presto / Trino

Presto (now Trino) is a **distributed SQL query engine** designed for **fast, interactive queries** across multiple data sources.

Key difference from Hive: Presto executes queries entirely **in memory** (no MapReduce) — results in seconds rather than minutes.

```
Hive query on 1 TB: ~15 minutes (MapReduce, disk I/O)
Presto query on 1 TB: ~30 seconds (in-memory, pipelined)
```

Presto can query multiple sources in one query:

```
-- Join Hive table (HDFS) with a MySQL table directly
SELECT h.user_id, h.total_spend, m.email
FROM hive.sales.orders h
JOIN mysql.users.accounts m ON h.user_id = m.id
WHERE h.year = 2024;
```

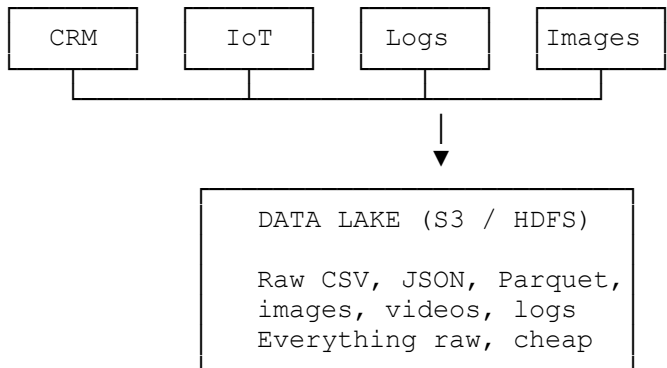
10. Data Lakes & Data Warehouses

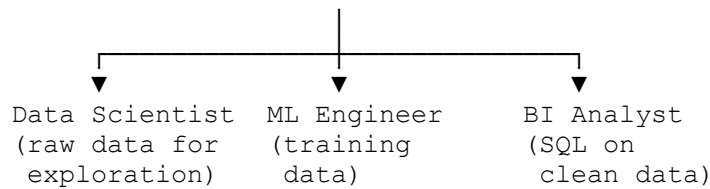
10.1 Data Lake

A **Data Lake** is a centralised repository that stores all structured, semi-structured, and unstructured data **in its raw, original format** until needed.

DATA LAKE:

Sources:





Schema-on-read: No schema enforced when storing data. Schema is applied when you read/query it. Flexible — you can store anything.

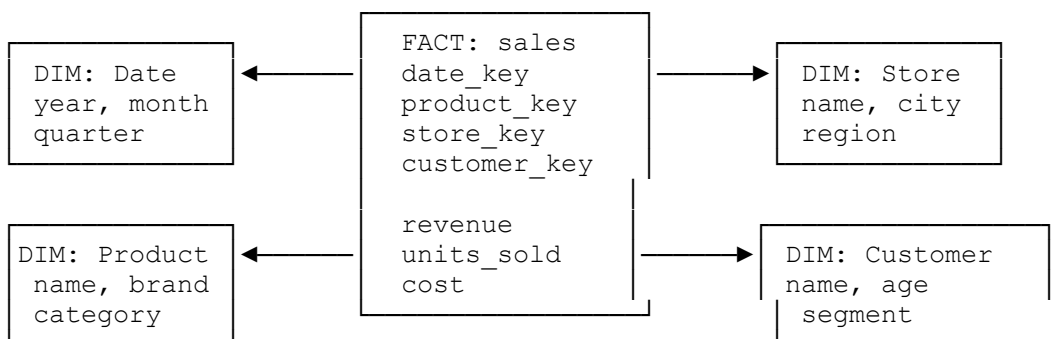
Risk — Data Swamp: Without proper metadata cataloguing and governance, a data lake becomes a "data swamp" where nobody knows what data exists, where it came from, or whether it can be trusted.

10.2 Data Warehouse

A **Data Warehouse** stores **cleaned, structured, and transformed data** optimised for business reporting and analytics. Data is processed (via ETL) before loading.

ETL PIPELINE:
 Source Systems
 (CRM, ERP, POS) → [Extract] → [Transform & Clean] → [Load] → Data Warehouse
 (fix errors, standardise, aggregate, join tables)
 (Snowflake, Redshift, BigQuery, Hive)

Data Warehouse Schema (Star Schema):



Schema-on-write: Schema strictly defined before data is loaded. Fast queries, governed, reliable.

10.3 Data Lake vs Data Warehouse

DATA LAKE vs DATA WAREHOUSE		
Aspect	Data Lake	Data Warehouse
Data type	All types (raw)	Structured only
Schema	Schema-on-read	Schema-on-write
Storage cost	Very cheap (S3/HDFS)	Expensive
Query performance	Slower	Fast (optimised)
Data quality	Raw, may be messy	Cleaned, reliable
Users	Data scientists	Business analysts
Purpose	Exploration, ML	Reporting, BI
Risk	Becomes data swamp	Too rigid, slow ETL

Modern solution — Data Lakehouse:

Combines the cheap storage and flexibility of a data lake with the reliability and performance of a data warehouse. Implemented using **Delta Lake**, **Apache Iceberg**, or **Apache Hudi** which add ACID transactions, schema enforcement, and fast queries on top of raw object storage (S3/HDFS).

11. Big Data Visualization

Data visualisation transforms processed big data into visual formats (charts, graphs, maps) that humans can understand and act on.

11.1 Challenges of Visualising Big Data

- **Volume:** Cannot plot 1 billion rows as individual dots — must aggregate first.
- **Real-time:** Dashboards must refresh as new data streams in.
- **Dimensionality:** High-dimensional data must be projected to 2D/3D for visualisation.
- **Variety:** Text, geospatial, time-series, graph data each need different chart types.

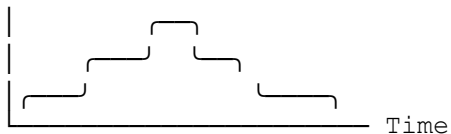
11.2 Common Tools

Tool	Best For	Description
Tableau	Business dashboards	Drag-and-drop BI, connects to most data sources
Power BI	Microsoft ecosystem	Excel-like BI with live dashboards
Kibana	Log/metrics analysis	Visualises Elasticsearch data, real-time dashboards

Apache Superset	Open-source BI	SQL-based dashboards, connects to Hive/Spark
D3.js	Custom web visualisations	JavaScript library for bespoke interactive charts
Grafana	Time-series metrics	Monitoring dashboards, operational data

11.3 Key Visualisation Types for Big Data

TIME SERIES: Line chart showing metric over time
[Revenue]



Best for: trends, seasonality, anomalies

GEOSPATIAL HEATMAP: Data intensity on a map
High concentration in NYC and LA (darker colours).
Best for: user distribution, delivery routes, store performance

HISTOGRAM: Distribution of a metric



Best for: understanding data distribution, outlier detection

WORD CLOUD: Frequency of text terms – larger = more frequent
Best for: trending topics, customer feedback themes

12. Machine Learning for Big Data

12.1 Why Standard ML Breaks at Scale

Traditional ML libraries (scikit-learn, standard NumPy) load the **entire dataset into one machine's memory**. At Big Data scale this is impossible:

```
Dataset: 500 GB of training data
scikit-learn: Needs 500 GB RAM on one machine → fails
Spark MLlib: Distributes data across 50 nodes (10 GB each) → works
```

Big Data ML frameworks solve this by distributing both the data AND the computation.

12.2 Spark MLlib

Spark MLlib is Spark's built-in distributed machine learning library. Algorithms are designed to work on RDDs/DataFrames split across a cluster.

Spark ML Pipeline:

Raw Data (distributed across cluster)

↓

PIPELINE (chain of transformations + model)

Stage 1: Tokenizer (text → words)

Stage 2: TF-IDF (words → numeric vectors)

Stage 3: StandardScaler (normalise features)

Stage 4: LogisticRegression (train classifier)

↓

Trained Model (can be saved and reused)

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.classification import RandomForestClassifier

# Feature assembly
assembler = VectorAssembler(
    inputCols=["age", "income", "credit_score"],
    outputCol="features"
)
scaler = StandardScaler(inputCol="features", outputCol="scaled_features")
rf = RandomForestClassifier(featuresCol="scaled_features",
labelCol="default", numTrees=100)

pipeline = Pipeline(stages=[assembler, scaler, rf])
model = pipeline.fit(train_df) # distributed training across cluster
predictions = model.transform(test_df) # distributed inference
```

Supported algorithms in MLlib:

- Classification: Logistic Regression, Decision Trees, Random Forest, SVM, Naive Bayes
- Regression: Linear Regression, Gradient Boosted Trees
- Clustering: K-Means, Gaussian Mixture Model, Bisecting K-Means
- Dimensionality Reduction: PCA, SVD
- Recommendation: ALS (Collaborative Filtering)
- NLP: TF-IDF, Word2Vec, Tokenizer, N-Gram

12.3 Scalable ML Algorithms

Linear Regression at Scale

Standard: load all data → solve normal equation → $O(\text{features}^3)$ computation

Distributed (Spark):

Each partition computes gradient for its portion of data

↓

Gradients combined (aggregated across all partitions)

↓

Model weights updated (one step of gradient descent)

↓

Repeat for N iterations ← data stays in memory across iterations (Spark advantage)

Logistic Regression at Scale

Same distributed gradient descent approach as linear regression. Each executor processes its partition, gradients are aggregated, weights updated. Spark keeps the model weights in driver memory (model is small) while data is distributed.

Random Forest at Scale

Build 100 trees in parallel:

Executor 1 → trains Tree 1, Tree 2, Tree 3

Executor 2 → trains Tree 4, Tree 5, Tree 6

...

Each tree uses a random sample of data (bootstrap sampling).

Final: collect all trees → ensemble prediction (majority vote).

K-Means Clustering at Scale

1. Choose K random initial centroids (on driver)
2. DISTRIBUTE centroids to all executors
3. Each executor assigns its points to nearest centroid → emits (centroid_id, point)
4. AGGREGATE: sum and count points per centroid → compute new centroid position
5. Check convergence: if centroids barely moved → stop, else go to step 2

Key: only centroids (small, $K \times \text{dimensions}$) need to be communicated.

Data points (large) stay distributed in place.

12.4 Model Evaluation & Tuning

Cross-Validation

Avoids the problem of getting lucky (or unlucky) with a single train/test split.

K-FOLD CROSS VALIDATION (K=5):

All data: [—————]

Fold 1: [TEST |————— TRAIN —————]

```

Fold 2:  [TRAIN| TEST |----- TRAIN -----]
Fold 3:  [TRAIN-----| TEST |----- TRAIN -----]
Fold 4:  [TRAIN-----| TEST |----- TRAIN -----]
Fold 5:  [TRAIN-----| TEST |----- TRAIN -----]

```

Average performance across 5 folds = more reliable estimate.
Used with Spark's CrossValidator class.

Hyperparameter Tuning

Model **parameters** are learned from data (e.g., weights in a model). **Hyperparameters** are set before training (e.g., number of trees, learning rate, max depth) and control how the model trains.

Grid Search:

Try every combination of hyperparameter values.

```

numTrees: [50, 100, 200]
maxDepth: [5, 10, 15]
Total combinations: 3 × 3 = 9 models to train and evaluate
Pick the combination that gives best validation accuracy.

```

In Spark:

```

from pyspark.ml.tuning import CrossValidator, ParamGridBuilder

paramGrid = ParamGridBuilder() \
    .addGrid(rf.numTrees, [50, 100, 200]) \
    .addGrid(rf.maxDepth, [5, 10, 15]) \
    .build()

cv = CrossValidator(estimator=pipeline,
                    estimatorParamMaps=paramGrid,
                    evaluator=evaluator,
                    numFolds=5)

best_model = cv.fit(train_df) # trains 9×5=45 models in parallel across
cluster

```

12.5 Deep Learning Frameworks for Big Data

Framework	Description	Use Case
TensorFlow	Google's deep learning library. Supports distributed training across GPUs/TPUs.	Image recognition, NLP, large neural networks
PyTorch	Facebook's deep learning library. Dynamic computation graph, Pythonic.	Research, NLP, computer vision
Horovod	Distributed training wrapper for TF/PyTorch. Scales to hundreds of GPUs.	Training large models on many GPUs

BigDL	Deep learning on Spark — runs TF/Keras on YARN clusters without GPUs.	Running DL on existing Hadoop infrastructure
--------------	---	--

Distributed Training Pattern:

DATA PARALLELISM (most common):

Model copied to each GPU/node.

Each node trains on a different subset of data.

After each batch: gradients averaged across all nodes → weights updated.

```

GPU 1: [batch 1] → gradient g1
GPU 2: [batch 2] → gradient g2
GPU 3: [batch 3] → gradient g3
GPU 4: [batch 4] → gradient g4

```

→ average(g1,g2,g3,g4) → update weights

4 GPUs → ~4x faster training (minus communication overhead)

Notes compiled: April 2026 | Subject: Big Data Analytics

Topic coverage aligned with GeeksforGeeks Big Data Analytics Tutorial

Covers: Big Data Foundations · Architecture · Distributed Computing · Hadoop

(HDFS/YARN/MapReduce) · Spark · Flink · Kafka · Sqoop · ZooKeeper · Hive · Pig · Presto ·

NoSQL (HBase, Cassandra, MongoDB) · Data Lakes & Warehouses · Big Data Visualisation ·

ML for Big Data (MLlib, Scalable Algorithms, Hyperparameter Tuning, Deep Learning)